

LlamaIndex

LlamaIndex is a popular LLM orchestration framework with a clean architecture and a focus on data structures and models. It integrates many LLMs as well as vector stores and other indexes and contains tooling for document loading (loader hub) and advanced RAG patterns.

LlamaIndex provides a lot of detailed examples for GenAI application development in their [blogs](#) and [documentation](#).

The Neo4j integration covers both the vector store as well as query generation from natural language and knowledge graph construction.

Functionality Includes

Neo4jVector

The Neo4j Vector integration supports a number of operations

- create vector from LlamaIndex documents
- query vector
- query vector with additional graph retrieval Cypher query
- construct vector instance from existing graph data
- hybrid search
- metadata filtering

```
%pip install llama-index-llms-openai
%pip install llama-index-vector-stores-neo4jvector
%pip install llama-index-embeddings-openai
%pip install neo4j

import os
import openai
from llama_index.vector_stores.neo4jvector import Neo4jVectorStore
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

os.environ["OPENAI_API_KEY"] = "OPENAI_API_KEY"
openai.api_key = os.environ["OPENAI_API_KEY"]
username = "neo4j"
password = "pleaseletmein"
url = "bolt://localhost:7687"
embed_dim = 1536

neo4j_vector = Neo4jVectorStore(username, password, url, embed_dim)
# load documents
documents = SimpleDirectoryReader("./data/paul_graham").load_data()
from llama_index.core import StorageContext

storage_context = StorageContext.from_defaults(vector_store=neo4j_vector)
index = VectorStoreIndex.from_documents(
```

```

        documents, storage_context=storage_context
    )

    query_engine = index.as_query_engine()
    response = query_engine.query("What happened at interleaf?")

```

Hybrid search

Hybrid search combines vector search with fulltext search with re-ranking and de-duplication of the results.

```

neo4j_vector_hybrid = Neo4jVectorStore(
    username, password, url, embed_dim, hybrid_search=True
)

storage_context = StorageContext.from_defaults(
    vector_store=neo4j_vector_hybrid
)
index = VectorStoreIndex.from_documents(
    documents, storage_context=storage_context
)
query_engine = index.as_query_engine()
response = query_engine.query("What happened at interleaf?")

```

Metadata filtering

Metadata filtering enhances vector search by allowing searches to be refined based on specific node properties. This integrated approach ensures more precise and relevant search results by leveraging both the vector similarities and the contextual attributes of the nodes.

```

from llama_index.core.vector_stores import (
    MetadataFilter,
    MetadataFilters,
    FilterOperator,
)

filters = MetadataFilters(
    filters=[
        MetadataFilter(
            key="theme", operator=FilterOperator.EQ, value="Fiction"
        ),
    ]
)

retriever = index.as_retriever(filters=filters)
retriever.retrieve("What is inception about?")

```

Neo4jPropertyGraphStore

The Neo4j Property Graph Store integration is a wrapper for the Neo4j Python driver. It allows querying and updating the Neo4j database in a simplified manner from LlamaIndex. Many integrations allow you to use the Neo4j Property Graph Store as a source of data for LlamaIndex.

Property graph index

Knowledge graph index can be used to extract graph representation of information from text and use it to construct a knowledge graph. The graph information can then be retrieved in a RAG application for more accurate responses.

```
%pip install llama-index llama-index-graph-stores-neo4j

from llama_index.core import SimpleDirectoryReader
from llama_index.core import PropertyGraphIndex
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI
from llama_index.core.indices.property_graph import SchemaLLMPPathExtractor

from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore

documents = SimpleDirectoryReader("./data/paul_graham/").load_data()
graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="password",
    url="bolt://localhost:7687",
)
# Extract graph from documents
index = PropertyGraphIndex.from_documents(
    documents,
    embed_model=OpenAIEmbedding(model_name="text-embedding-3-small"),
    kg_extractors=[
        SchemaLLMPPathExtractor(
            llm=OpenAI(model="gpt-3.5-turbo", temperature=0.0)
        )
    ],
    property_graph_store=graph_store,
    show_progress=True,
)

# Define retriever
retriever = index.as_retriever(
    include_text=False, # include source text in returned nodes, default
True
)
results = retriever.retrieve("What happened at Interleaf and Viaweb?")
for record in results:
    print(record.text)

# Question answering
query_engine = index.as_query_engine(include_text=True)
```

```
response = query_engine.query("What happened at Interleaf and Viaweb?")
print(str(response))
```

Property Graph constructing modules

LlamaIndex features multiple graph construction modules. Property graph construction in LlamaIndex works by performing a series of `kg_extractors` on each text chunk, and attaching entities and relations as metadata to each llama-index node. You can use as many as you like here, and they will all get applied. Learn more about them in the [documentation](#).

Here is an example of graph construction using a predefined schema.

```
%pip install llama-index llama-index-graph-stores-neo4j
```

```
from typing import Literal
from llama_index.core import SimpleDirectoryReader
from llama_index.core import PropertyGraphIndex
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI
from llama_index.core.indices.property_graph import SchemaLLMPathExtractor

from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore

# best practice to use upper-case
entities = Literal["PERSON", "PLACE", "ORGANIZATION"]
relations = Literal["HAS", "PART_OF", "WORKED_ON", "WORKED_WITH",
"WORKED_AT"]

# define which entities can have which relations
validation_schema = {
    "PERSON": ["HAS", "PART_OF", "WORKED_ON", "WORKED_WITH", "WORKED_AT"],
    "PLACE": ["HAS", "PART_OF", "WORKED_AT"],
    "ORGANIZATION": ["HAS", "PART_OF", "WORKED_WITH"],
}

kg_extractor = SchemaLLMPathExtractor(
    llm=OpenAI(model="gpt-3.5-turbo", temperature=0.0),
    possible_entities=entities,
    possible_relations=relations,
    kg_validation_schema=validation_schema,
    # if false, allows for values outside of the schema
    # useful for using the schema as a suggestion
    strict=True,
)

graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="password",
    url="bolt://localhost:7687",
)

documents = SimpleDirectoryReader("./data/paul_graham/").load_data()
index = PropertyGraphIndex.from_documents(
    documents,
```

```

kg_extractors=[kg_extractor],
embed_model=OpenAIEmbedding(model_name="text-embedding-3-small"),
property_graph_store=graph_store,
show_progress=True,
)

```

Property graph querying modules

Labeled property graphs can be queried in several ways to retrieve nodes and paths. And in LlamaIndex, you can combine several node retrieval methods at once! Learn more about which ones are available in the [documentation](#).

You can also define a custom graph retriever as shown below.

```

from llama_index.core.retrievers import (
    CustomPGRetriever,
    VectorContextRetriever,
    TextToCypherRetriever,
)
from llama_index.core.query_engine import RetrieverQueryEngine
from llama_index.core.graph_stores import PropertyGraphStore
from llama_index.core.vector_stores.types import VectorStore
from llama_index.core.embeddings import BaseEmbedding
from llama_index.core.prompts import PromptTemplate
from llama_index.core.llms import LLM
from llama_index.postprocessor.cohere_rerank import CohereRerank

```

```

from typing import Optional, Any, Union

```

```

class MyCustomRetriever(CustomPGRetriever):
    """Custom retriever with cohere reranking."""

    def init(
        self,
        ## vector context retriever params
        embed_model: Optional[BaseEmbedding] = None,
        vector_store: Optional[VectorStore] = None,
        similarity_top_k: int = 4,
        path_depth: int = 1,
        ## text-to-cypher params
        llm: Optional[LLM] = None,
        text_to_cypher_template: Optional[Union[PromptTemplate, str]] =
None,
        ## cohere reranker params
        cohere_api_key: Optional[str] = None,
        cohere_top_n: int = 2,
        **kwargs: Any,
    ) -> None:
        """Uses any kwargs passed in from class constructor."""

        self.vector_retriever = VectorContextRetriever(

```

```

        self.graph_store,
        include_text=self.include_text,
        embed_model=embed_model,
        vector_store=vector_store,
        similarity_top_k=similarity_top_k,
        path_depth=path_depth,
    )

```

```

    self.cypher_retriever = TextToCypherRetriever(
        self.graph_store,
        llm=llm,
        text_to_cypher_template=text_to_cypher_template
        ## NOTE: you can attach other parameters here if you'd like
    )

```

```

    self.reranker = CohereRerank(
        api_key=cohere_api_key, top_n=cohere_top_n
    )

```

```

def custom_retrieve(self, query_str: str) -> str:
    """Define custom retriever with reranking.

```

Could return `str`, `TextNode`, `NodeWithScore`, or a list of those.

```

    """
    nodes_1 = self.vector_retriever.retrieve(query_str)
    nodes_2 = self.cypher_retriever.retrieve(query_str)
    reranked_nodes = self.reranker.postprocess_nodes(
        nodes_1 + nodes_2, query_str=query_str
    )

```

```

    ## TMP: please change
    final_text = "\n\n".join(
        [n.get_content(metadata_mode="llm") for n in reranked_nodes]
    )

```

```

    return final_text

```

```

custom_sub_retriever = MyCustomRetriever(
    index.property_graph_store,
    include_text=True,
    vector_store=index.vector_store,
    cohere_api_key="...",
)

```

```

query_engine = RetrieverQueryEngine.from_args(
    index.as_retriever(sub_retrievers=[custom_sub_retriever]), llm=llm
)

```

```

response = query_engine.query("Did the author like programming?")
print(str(response))

```

Using a Property Graph Index

A property graph is a knowledge collection of labeled nodes (i.e. entity categories, text labels, etc.) with properties (i.e. metadata), linked together by relationships into structured paths.

In LlamaIndex, the `PropertyGraphIndex` provides key orchestration around

- constructing a graph
- querying a graph

Usage

Basic usage can be found by simply importing the class and using it:

```
from llama_index.core import PropertyGraphIndex

# create
index = PropertyGraphIndex.from_documents(
    documents,
)

# use
retriever = index.as_retriever(
    include_text=True, # include source chunk with matching paths
    similarity_top_k=2, # top k for vector kg node retrieval
)
nodes = retriever.retrieve("Test")

query_engine = index.as_query_engine(
    include_text=True, # include source chunk with matching paths
    similarity_top_k=2, # top k for vector kg node retrieval
)
response = query_engine.query("Test")

# save and load
index.storage_context.persist(persist_dir="./storage")

from llama_index.core import StorageContext, load_index_from_storage

index = load_index_from_storage(
    StorageContext.from_defaults(persist_dir="./storage")
)

# loading from existing graph store (and optional vector store)
# load from existing graph/vector store
index = PropertyGraphIndex.from_existing(
    property_graph_store=graph_store, vector_store=vector_store, ...
)
```

Construction

Property graph construction in LlamaIndex works by performing a series of `kg_extractors` on each chunk, and attaching entities and relations as metadata to each llama-index node. You can use as many as you like here, and they will all get applied.

If you've used transformations or metadata extractors with the [ingestion pipeline](#), then this will be very familiar (and these `kg_extractors` are compatible with the ingestion pipeline)!

Extractors are set using the appropriate kwarg:

```
index = PropertyGraphIndex.from_documents(  
    documents,  
    kg_extractors=[extractor1, extractor2, ...],  
)  
  
# insert additional documents / nodes  
index.insert(document)  
index.insert_nodes(nodes)
```

If not provided, the defaults are `SimpleLLMPPathExtractor` and `ImplicitPathExtractor`.

All `kg_extractors` are detailed below.

(default) SimpleLLMPPathExtractor

Extract short statements using an LLM to prompt and parse single-hop paths in the format (entity1, relation, entity2)

```
from llama_index.core.indices.property_graph import SimpleLLMPPathExtractor  
  
kg_extractor = SimpleLLMPPathExtractor(  
    llm=llm,  
    max_paths_per_chunk=10,  
    num_workers=4,  
    show_progress=False,  
)
```

If you want, you can also customize the prompt and the function used to parse the paths.

Here's a simple (but naive) example:

```
prompt = (  
    "Some text is provided below. Given the text, extract up to "  
    "{max_paths_per_chunk} "  
    "knowledge triples in the form of `subject,predicate,object` on each line. Avoid stopwords.\n"  
)
```



```
def parse_fn(response_str: str) -> List[Tuple[str, str, str]]:
    lines = response_str.split("\n")
    triples = [line.split(",") for line in lines]
    return triples
```

```
kg_extractor = SimpleLLMPathExtractor(
    llm=llm,
    extract_prompt=prompt,
    parse_fn=parse_fn,
)
(default) ImplicitPathExtractor
```

Extract paths using the `node.relationships` attribute on each llama-index node object.

This extractor does not need an LLM or embedding model to run, since it's merely parsing properties that already exist on llama-index node objects.

```
from llama_index.core.indices.property_graph import ImplicitPathExtractor
```

```
kg_extractor = ImplicitPathExtractor()
DynamicLLMPathExtractor
```

Will extract paths (including entity types!) according to optional list of allowed entity types and relation types. If none are provided, then the LLM will assign types as it sees fit. If they are provided, it will help guide the LLM, but will not enforce exactly those types.

```
from llama_index.core.indices.property_graph import DynamicLLMPathExtractor
```

```
kg_extractor = DynamicLLMPathExtractor(
    llm=llm,
    max_triplets_per_chunk=20,
    num_workers=4,
    allowed_entity_types=["POLITICIAN", "POLITICAL_PARTY"],
    allowed_relation_types=["PRESIDENT_OF", "MEMBER_OF"],
)
SchemaLLMPathExtractor
```

Extract paths following a strict schema of allowed entities, relationships, and which entities can be connected to which relationships.

Using pydantic, structured outputs from LLMs, and some clever validation, we can dynamically specify a schema and verify the extractions per-path.

```
from typing import Literal
from llama_index.core.indices.property_graph import SchemaLLMPathExtractor
```

```
# recommended uppercase, underscore separated
entities = Literal["PERSON", "PLACE", "THING"]
relations = Literal["PART_OF", "HAS", "IS_A"]
```

```

schema = {
    "PERSON": ["PART_OF", "HAS", "IS_A"],
    "PLACE": ["PART_OF", "HAS"],
    "THING": ["IS_A"],
}

kg_extractor = SchemaLLMPathExtractor(
    llm=llm,
    possible_entities=entities,
    possible_relations=relations,
    kg_validation_schema=schema,
    strict=True, # if false, will allow triplets outside of the schema
    num_workers=4,
    max_triplets_per_chunk=10,
)

```

This extractor is extremely customizable, and has options to customize

- various aspects of the schema (as seen above)
- the `extract_prompt`
- `strict=False` vs. `strict=True`, to allow triples outside of the schema or not
- passing in your own custom `kg_schema_cls` if you are a pydantic pro and wanted to create your own pydantic class with custom validation.

Retrieval and Querying

Labeled property graphs can be queried in several ways to retrieve nodes and paths. And in LlamaIndex, we can combine several node retrieval methods at once!

```

# create a retriever
retriever = index.as_retriever(sub_retrievers=[retriever1, retriever2, ...])

# create a query engine
query_engine = index.as_query_engine(
    sub_retrievers=[retriever1, retriever2, ...]
)

```

If no sub-retrievers are provided, the defaults are `LLMSynonymRetriever` and `VectorContextRetriever` (if embeddings are enabled).

All retrievers currently include:

- `LLMSynonymRetriever` - retrieve based on LLM generated keywords/synonyms
- `VectorContextRetriever` - retrieve based on embedded graph nodes
- `TextToCypherRetriever` - ask the LLM to generate cypher based on the schema of the property graph
- `CypherTemplateRetriever` - use a cypher template with params inferred by the LLM
- `CustomPGRetriever` - easy to subclass and implement custom retrieval logic

Generally, you would define one or more of these sub-retrievers and pass them to the PGRetriever:

```
from llama_index.core.indices.property_graph import (
    PGRetriever,
    VectorContextRetriever,
    LLMSynonymRetriever,
)

sub_retrievers = [
    VectorContextRetriever(index.property_graph_store, ...),
    LLMSynonymRetriever(index.property_graph_store, ...),
]

retriever = PGRetriever(sub_retrievers=sub_retrievers)

nodes = retriever.retrieve("<query>")
```

Read on below for more details on all retrievers.

(default) LLMSynonymRetriever

The LLMSynonymRetriever takes the query, and tries to generate keywords and synonyms to retrieve nodes (and therefore the paths connected to those nodes).

Explicitly declaring the retriever allows you to customize several options. Here are the defaults:

```
from llama_index.core.indices.property_graph import LLMSynonymRetriever

prompt = (
    "Given some initial query, generate synonyms or related keywords up to {max_keywords} in total, "
    "considering possible cases of capitalization, pluralization, common expressions, etc.\n"
    "Provide all synonyms/keywords separated by '^' symbols: 'keyword1^keyword2^...'\n"
    "Note, result should be in one-line, separated by '^' symbols."
    "----\n"
    "QUERY: {query_str}\n"
    "----\n"
    "KEYWORDS: "
)

def parse_fn(self, output: str) -> list[str]:
    matches = output.strip().split("^")

    # capitalize to normalize with ingestion
    return [x.strip().capitalize() for x in matches if x.strip()]

synonym_retriever = LLMSynonymRetriever(
    index.property_graph_store,
    llm=llm,
    # include source chunk text with retrieved paths
    include_text=False,
    synonym_prompt=prompt,
```

```

    output_parsing_fn=parse_fn,
    max_keywords=10,
    # the depth of relations to follow after node retrieval
    path_depth=1,
)

retriever = index.as_retriever(sub_retrievers=[synonym_retriever])
(default, if supported) VectorContextRetriever

```

The `VectorContextRetriever` retrieves nodes based on their vector similarity, and then fetches the paths connected to those nodes.

If your graph store supports vectors, then you only need to manage that graph store for storage. Otherwise, you will need to provide a vector store in addition to the graph store (by default, uses the in-memory `SimpleVectorStore`).

```

from llama_index.core.indices.property_graph import VectorContextRetriever

vector_retriever = VectorContextRetriever(
    index.property_graph_store,
    # only needed when the graph store doesn't support vector queries
    # vector_store=index.vector_store,
    embed_model=embed_model,
    # include source chunk text with retrieved paths
    include_text=False,
    # the number of nodes to fetch
    similarity_top_k=2,
    # the depth of relations to follow after node retrieval
    path_depth=1,
    # can provide any other kwargs for the VectorStoreQuery class
    ...,
)

retriever = index.as_retriever(sub_retrievers=[vector_retriever])
TextToCypherRetriever

```

The `TextToCypherRetriever` uses a graph store schema, your query, and a prompt template for text-to-cypher in order to generate and execute a cypher query.

NOTE: Since the `SimplePropertyGraphStore` is not actually a graph database, it does not support cypher queries.

You can inspect the schema by using `index.property_graph_store.get_schema_str()`.

```

from llama_index.core.indices.property_graph import TextToCypherRetriever

DEFAULT_RESPONSE_TEMPLATE = (
    "Generated Cypher query:\n{query}\n\n" "Cypher Response:\n{response}"
)
DEFAULT_ALLOWED_FIELDS = ["text", "label", "type"]

DEFAULT_TEXT_TO_CYPHER_TEMPLATE = (
    index.property_graph_store.text_to_cypher_template,

```

```
)
```

```
cypher_retriever = TextToCypherRetriever(  
    index.property_graph_store,  
    # customize the LLM, defaults to Settings.llm  
    llm=llm,  
    # customize the text-to-cypher template.  
    # Requires `schema` and `question` template args  
    text_to_cypher_template=DEFAULT_TEXT_TO_CYPHER_TEMPLATE,  
    # customize how the cypher result is inserted into  
    # a text node. Requires `query` and `response` template args  
    response_template=DEFAULT_RESPONSE_TEMPLATE,  
    # an optional callable that can clean/verify generated cypher  
    cypher_validator=None,  
    # allowed fields in the resulting  
    allowed_output_field=DEFAULT_ALLOWED_FIELDS,  
)
```

NOTE: Executing arbitrary cypher has its risks. Ensure you take the needed measures (read-only roles, sandboxed env, etc.) to ensure safe usage in a production environment.

CypherTemplateRetriever

This is a more constrained version of the `TextToCypherRetriever`. Rather than letting the LLM have free-range of generating any cypher statement, we can instead provide a cypher template and have the LLM fill in the blanks.

To illustrate how this works, here is a small example:

```
# NOTE: current v1 is needed  
from pydantic import BaseModel, Field  
from llama_index.core.indices.property_graph import CypherTemplateRetriever  
  
# write a query with template params  
cypher_query = """  
MATCH (c:Chunk)-[:MENTIONS]->(o)  
WHERE o.name IN $names  
RETURN c.text, o.name, o.label;  
"""  
  
# create a pydantic class to represent the params for our query  
# the class fields are directly used as params for running the cypher query  
class TemplateParams(BaseModel):  
    """Template params for a cypher query."""  
  
    names: list[str] = Field(  
        description="A list of entity names or keywords to use for lookup in a knowledge graph."  
    )  
  
template_retriever = CypherTemplateRetriever(  
    index.property_graph_store, TemplateParams, cypher_query
```

) Storage

Currently, supported graph stores for property graphs include:

	In-Memory	Native Embedding Support	Async	Server or disk based?
SimplePropertyGraphStore	✓	✗	✗	Disk
Neo4jPropertyGraphStore	✗	✓	✗	Server
NebulaPropertyGraphStore	✗	✗	✗	Server
TiDBPropertyGraphStore	✗	✓	✗	Server
FalkorDBPropertyGraphStore	✗	✓	✗	Server

Saving to/from disk

The default property graph store, `SimplePropertyGraphStore`, stores everything in memory and persists and loads from disk.

Here's an example of saving/loading an index with the default graph store:

```
from llama_index.core import StorageContext, load_index_from_storage
from llama_index.core.indices import PropertyGraphIndex

# create
index = PropertyGraphIndex.from_documents(documents)

# save
index.storage_context.persist("./storage")

# load
storage_context = StorageContext.from_defaults(persist_dir="./storage")
index = load_index_from_storage(storage_context)
```

Saving and Loading with Integrations

Integrations typically save automatically. Some graph stores will support vectors, others might not. You can always combine a graph store with an external vector db as well.

This example shows how you might save/load a property graph index using Neo4j and Qdrant.

Note: If qdrant wasn't passed in, neo4j would store and use the embeddings on its own. This example illustrates the flexibility beyond that.

```
pip install llama-index-graph-stores-neo4j llama-index-vector-stores-qdrant
```

```
from llama_index.core import StorageContext, load_index_from_storage
from llama_index.core.indices import PropertyGraphIndex
from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore
from llama_index.vector_stores.qdrant import QdrantVectorStore
from qdrant_client import QdrantClient, AsyncQdrantClient
```

```
vector_store = QdrantVectorStore(
    "graph_collection",
    client=QdrantClient(...),
    aclient=AsyncQdrantClient(...),
)
```

```
graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="<password>",
    url="bolt://localhost:7687",
)
```

```
# creates an index
index = PropertyGraphIndex.from_documents(
    documents,
    property_graph_store=graph_store,
    # optional, neo4j also supports vectors directly
    vector_store=vector_store,
    embed_kg_nodes=True,
)
```

```
# load from existing graph/vector store
index = PropertyGraphIndex.from_existing(
    property_graph_store=graph_store,
    # optional, neo4j also supports vectors directly
    vector_store=vector_store,
    embed_kg_nodes=True,
)
```

Using the Property Graph Store Directly

The base storage class for property graphs is the `PropertyGraphStore`. These property graph stores are constructed using different types of `LabeledNode` objects, and connected using `Relation` objects.

We can create these ourselves, and also insert ourselves!

```
from llama_index.core.graph_stores import (
    SimplePropertyGraphStore,
    EntityNode,
    Relation,
)
from llama_index.core.schema import TextNode

graph_store = SimplePropertyGraphStore()
```

```

entities = [
    EntityNode(name="llama", label="ANIMAL", properties={"key": "val"}),
    EntityNode(name="index", label="THING", properties={"key": "val"}),
]

relations = [
    Relation(
        label="HAS",
        source_id=entities[0].id,
        target_id=entities[1].id,
        properties={},
    )
]

graph_store.upsert_nodes(entities)
graph_store.upsert_relations(relations)

# optionally, we can also insert text chunks
source_chunk = TextNode(id_="source", text="My llama has an index.")

# create relation for each of our entities
source_relations = [
    Relation(
        label="HAS_SOURCE",
        source_id=entities[0].id,
        target_id="source",
    ),
    Relation(
        label="HAS_SOURCE",
        source_id=entities[1].id,
        target_id="source",
    ),
]
graph_store.upsert_llama_nodes([source_chunk])
graph_store.upsert_relations(source_relations)

```

Other helpful methods on the graph store include:

- `graph_store.get(ids=[])` - gets nodes based on ids
- `graph_store.get(properties={"key": "val"})` - gets nodes based on matching properties
- `graph_store.get_rel_map([entity_node], depth=2)` - gets triples up to a certain depth
- `graph_store.get_llama_nodes(['id1'])` - gets the original text nodes
- `graph_store.delete(ids=['id1'])` - delete based on ids
- `graph_store.delete(properties={"key": "val"})` - delete based on properties
- `graph_store.structured_query("<cypher query>")` - runs a cypher query (assuming the graph store supports it)

In addition a versions exist for all of these for async support (i.e. `aget`, `adelete`, etc.).

Advanced Customization

As with all components in LlamaIndex, you can sub-class modules and customize things to work exactly as you need, or try out new ideas and research new modules!

Sub-Classing Extractors

Graph extractors in LlamaIndex subclass the TransformComponent class. If you've worked with the ingestion pipeline before, this will be familiar since it is the same class.

The requirement for extractors is that the insert graph data into the metadata of the node, which will then be processed later on by the index.

Here is a small example of sub-classing to create a custom extractor:

```
from llama_index.core.graph_store.types import (
    EntityNode,
    Relation,
    KG_NODES_KEY,
    KG_RELATIONS_KEY,
)
from llama_index.core.schema import BaseNode, TransformComponent

class MyGraphExtractor(TransformComponent):
    # the init is optional
    # def __init__(self, ...):
    #     ...

    def __call__(
        self, llama_nodes: list[BaseNode], **kwargs
    ) -> list[BaseNode]:
        for llama_node in llama_nodes:
            # be sure to not overwrite existing entities/relations

            existing_nodes = llama_node.metadata.pop(KG_NODES_KEY, [])
            existing_relations = llama_node.metadata.pop(KG_RELATIONS_KEY, [])

            existing_nodes.append(
                EntityNode(
                    name="llama", label="ANIMAL", properties={"key": "val"}
                )
            )
            existing_nodes.append(
                EntityNode(
                    name="index", label="THING", properties={"key": "val"}
                )
            )

            existing_relations.append(
                Relation(
                    label="HAS",
                    source_id="llama",
```

```

        target_id="index",
        properties={},
    )
)

# add back to the metadata

llama_node.metadata[KG_NODES_KEY] = existing_nodes
llama_node.metadata[KG_RELATIONS_KEY] = existing_relations

return llama_nodes

# optional async method
# async def acall(self, llama_nodes: list[BaseNode], **kwargs) -> list[BaseNode]:
# ...

```

Sub-Classing Retrievers

The retriever is a bit more complicated than the extractors, and has it's own special class to help make sub-classing easier.

The return type of the retrieval is extremely flexible. It could be

- a string
- a TextNode
- a NodeWithScore
- a list of one of the above

Here is a small example of sub-classing to create a custom retriever:

```

from llama_index.core.indices.property_graph import (
    CustomPGRetriever,
    CUSTOM_RETRIEVE_TYPE,
)

class MyCustomRetriever(CustomPGRetriever):
    def init(self, my_option_1: bool = False, **kwargs) -> None:
        """Uses any kwargs passed in from class constructor."""
        self.my_option_1 = my_option_1
        # optionally do something with self.graph_store

    def custom_retrieve(self, query_str: str) -> CUSTOM_RETRIEVE_TYPE:
        # some some operation with self.graph_store
        return "result"

# optional async method
# async def acustom_retrieve(self, query_str: str) -> str:
# ...

custom_retriever = MyCustomRetriever(graph_store, my_option_1=True)

retriever = index.as_retriever(sub_retrievers=[custom_retriever])

```

For more complicated customization and use-cases, it is recommended to check out the source code and directly sub-class `BasePGRetriever`.

Property Graph Construction with Predefined Schemas

In this notebook, we walk through using Neo4j, Ollama and Huggingface to build a property graph.

Specifically, we will be using the `SchemaLLMPathExtractor` which allows us to specify an exact schema containing possible entity types, relation types, and defining how they can be connected together.

This is useful for when you have a specific graph you want to build, and want to limit what the LLM is predicting.

```
%pip install llama-index
%pip install llama-index-llms-ollama
%pip install llama-index-embeddings-huggingface
# Optional
%pip install llama-index-graph-stores-neo4j
%pip install llama-index-graph-stores-nebula
```

Load Data

First, lets download some sample data to play with.

```
!mkdir -p 'data/paul_graham/'
!wget 'https://raw.githubusercontent.com/run-llama/llama_index/main/docs/examples/data/paul_graham/paul_graham_essay.txt' -O
'data/paul_graham/paul_graham_essay.txt'
--2024-06-26 11:12:16-- https://raw.githubusercontent.com/run-llama/llama_index/main/docs/examples/data/paul_graham/paul_graham_essay.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133,
185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 75042 (73K) [text/plain]
Saving to: 'data/paul_graham/paul_graham_essay.txt'

data/paul_graham/pa 100%[=====>] 73.28K --KB/s in 0.007s

2024-06-26 11:12:16 (10.4 MB/s) - 'data/paul_graham/paul_graham_essay.txt' saved [75042/75042]
from llama_index.core import SimpleDirectoryReader

documents = SimpleDirectoryReader("./data/paul_graham/").load_data()
```

Graph Construction

To construct our graph, we are going to take advantage of the SchemaLLMPathExtractor to construct our graph.

Given some schema for a graph, we can extract entities and relations that follow this schema, rather than letting the LLM decide entities and relations at random.

```
import nest_asyncio

nest_asyncio.apply()
from typing import Literal
from llama_index.llms.ollama import Ollama
from llama_index.core.indices.property_graph import SchemaLLMPathExtractor

# best practice to use upper-case
entities = Literal["PERSON", "PLACE", "ORGANIZATION"]
relations = Literal["HAS", "PART_OF", "WORKED_ON", "WORKED_WITH", "WORKED_AT"]

# define which entities can have which relations
validation_schema = {
    "PERSON": ["HAS", "PART_OF", "WORKED_ON", "WORKED_WITH", "WORKED_AT"],
    "PLACE": ["HAS", "PART_OF", "WORKED_AT"],
    "ORGANIZATION": ["HAS", "PART_OF", "WORKED_WITH"],
}

kg_extractor = SchemaLLMPathExtractor(
    llm=Ollama(
        model="llama3",
        json_mode=True,
        request_timeout=3600,
        # Manually set the context window to limit memory usage
        context_window=8000,
    ),
    possible_entities=entities,
    possible_relations=relations,
    kg_validation_schema=validation_schema,
    # if false, allows for values outside of the schema
    # useful for using the schema as a suggestion
    strict=True,
)
```

Now, You can use SimplePropertyGraph, Neo4j, or NebulaGraph to store the graph.

Option 1. Neo4j

To launch Neo4j locally, first ensure you have docker installed. Then, you can launch the database with the following docker command

Terminal window

```
docker run \
  -p 7474:7474 -p 7687:7687 \
  -v $PWD/data:/data -v $PWD/plugins:/plugins \
```

```
--name neo4j-apoc \
-e NEO4J_apoc_export_file_enabled=true \
-e NEO4J_apoc_import_file_enabled=true \
-e NEO4J_apoc_import_file_use__neo4j__config=true \
-e NEO4JLABS_PLUGINS=[\"apoc\"] \
neo4j:latest
```

From here, you can open the db at <http://localhost:7474/>. On this page, you will be asked to sign in. Use the default username/password of neo4j and neo4j.

Once you login for the first time, you will be asked to change the password.

After this, you are ready to create your first property graph!

```
from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore

graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="<password>",
    url="bolt://localhost:7687",
)
vec_store = None
```

Option 2. NebulaGraph

To launch NebulaGraph locally, first ensure you have docker installed. Then, you can launch the database with the following docker command.

Terminal window

```
mkdir nebula-docker-compose
cd nebula-docker-compose
curl --output docker-compose.yaml https://raw.githubusercontent.com/vesoft-inc/nebula-docker-compose/master/docker-compose-lite.yaml
docker compose up
```

After this, you are ready to create your first property graph!

Other options/details for deploying NebulaGraph can be found in the [docs](#):

- [ad-hoc cluster in Google Colab](#).
- [Docker Desktop Extension](#).

```
from llama_index.graph_stores.nebula import NebulaPropertyGraphStore
from llama_index.core.vector_stores.simple import SimpleVectorStore

graph_store = NebulaPropertyGraphStore(
    space="llamaindex_nebula_property_graph", overwrite=True
)
vec_store = SimpleVectorStore()
```

If you want to explore the graph with NebulaGraph Jupyter extension, run the following commands. Or just skip them.

```
%pip install jupyter-nebulagraph
# load NebulaGraph Jupyter extension to enable %ngql magic
%load_ext ngql
# connect to NebulaGraph service
%ngql --address 127.0.0.1 --port 9669 --user root --password nebula
%ngql CREATE SPACE IF NOT EXISTS
llamaindex_nebula_property_graph(vid_type=FIXED_STRING(256));
# use the graph space, which is similar to "use database" in MySQL
# The space was created in async way, so we need to wait for a while before using it, retry it if failed
%ngql USE llamaindex_nebula_property_graph;
```

Start building!

NOTE: Using a local model will be slower when extracting compared to API based models. Local models (like Ollama) are typically limited to sequential processing. Expect this to take about 10 minutes on an M2 Max.

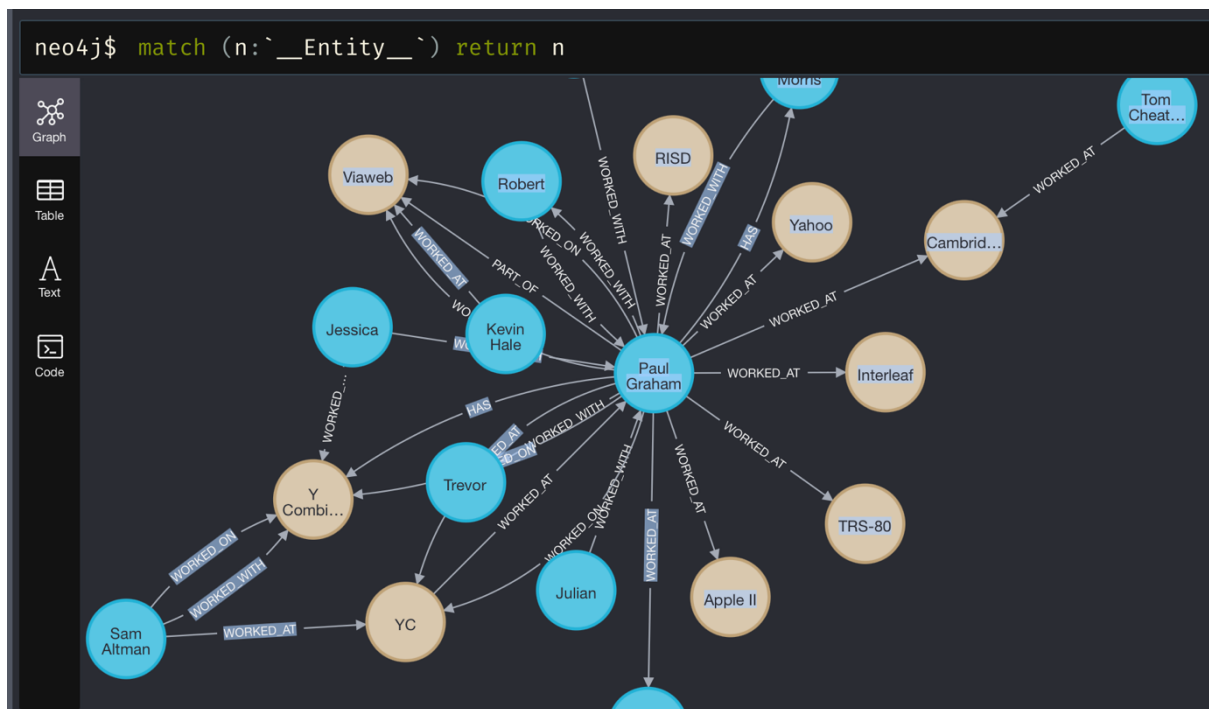
```
from llama_index.core import PropertyGraphIndex
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

index = PropertyGraphIndex.from_documents(
    documents,
    kg_extractors=[kg_extractor],
    embed_model=HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5"),
    property_graph_store=graph_store,
    vector_store=vec_store,
    show_progress=True,
)
```

If we inspect the graph created, we can see that it only includes the relations and entity types that we defined!

```
# If using NebulaGraph Jupyter extension
%ngql MATCH p=()-[]->() RETURN p LIMIT 20;
%ng_draw
```

Or Neo4j:



For information on all kg_extractors, see [the documentation](#).

Querying

Now that our graph is created, we can query it.

As is the theme with this notebook, we will be using a lower-level API and constructing all our retrievers ourselves!

```
from llama_index.core.indices.property_graph import (
    LLMSynonymRetriever,
    VectorContextRetriever,
)
```

```
llm_synonym = LLMSynonymRetriever(
    index.property_graph_store,
    llm=Ollama(
        model="llama3",
        request_timeout=3600,
        # Manually set the context window to limit memory usage
        context_window=8000,
    ),
    include_text=False,
)
```

```
vector_context = VectorContextRetriever(
    index.property_graph_store,
    embed_model=HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5"),
    include_text=False,
)
```

```
retriever = index.as_retriever(
    sub_retrievers=[
```

```

        llm_synonym,
        vector_context,
    ]
)
nodes = retriever.retrieve("What happened at Interleaf?")

```

```

for node in nodes:
    print(node.text)
Interleaf -> HAS -> Paul Graham
Interleaf -> HAS -> Emacs
Interleaf -> HAS -> Release Engineering
Interleaf -> HAS -> Viaweb
Interleaf -> HAS -> Y Combinator
Interleaf -> HAS -> impressive technology
Interleaf -> HAS -> smart people

```

We can also create a query engine with similar syntax.

```

query_engine = index.as_query_engine(
    sub_retrievers=[
        llm_synonym,
        vector_context,
    ],
    llm=Ollama(
        model="llama3",
        request_timeout=3600,
        # Manually set the context window to limit memory usage
        context_window=8000,
    ),
)

response = query_engine.query("What happened at Interleaf?")

print(str(response))
Paul Graham worked there, as well as other smart people. Emacs was also present.

```